

PATHFINDER

AI-Powered Academic & Career Advising System

Integration Phase Blueprint

End-to-End Backbone: KG + RAG Orchestration

Document Type: Implementation Blueprint

Phase: Integration Phase — Two-Engine System

Status: Implementation-Ready

Audience: Team, AI Collaborators, Supervisor

Table of Contents

Table of Contents	2
1. How to Use This Document	4
2. Phase Goal & Scope.....	5
2.1 Goal Statement	5
2.2 MVP Definition.....	5
2.3 Scope	5
3. System Architecture.....	6
3.1 High-Level Architecture Diagram.....	6
3.2 Component Summary.....	6
4. Supported User Workflows	8
5. Component Specifications	9
5.1 UI / Client	9
5.2 Gateway / Backend API.....	9
5.3 Session & Context Manager	9
5.4 Student Context Provider	10
5.5 Query Understanding Layer.....	10
5.6 Orchestrator (Workflow Coordinator).....	11
5.7 Engine Wrappers (KG & RAG)	11
5.8 Response Composer.....	12
6. Data Contracts.....	13
6.1 API Request & Response	13
Incoming Request (UI → Gateway).....	13
Outgoing Response (Gateway → UI).....	13
6.2 Structured Query (Query Understanding → Orchestrator)	13
6.3 Student Context (Provider → Session Manager → Orchestrator)	13
6.4 Session State (Stored by Session Manager)	14
6.5 Aggregated Result Package (Orchestrator → Response Composer)	15
7. Curated Test Student Profile.....	16
7.1 Profile Design Requirements	16
7.2 Profile JSON Structure	16
8. Runtime Query Flow	18
8.1 Phase Overview	18
8.2 Detailed End-to-End Flow Trace.....	18
8.3 Error Handling Rules	20
9. Implementation Order	21

10. Testing Strategy.....	23
10.1 Component-Level Tests	23
10.2 End-to-End Acceptance Tests	23
11. Deployment Strategy	25
11.1 Guiding Principles	25
11.2 What Is and Is Not Containerized	25
11.3 Service Map	25
11.4 Docker Compose Structure	26
11.5 Environment Variables	27
11.6 Startup & Teardown.....	27
11.7 Recommended Project Structure.....	27
12. Open Decisions (To Be Made During Implementation).....	29

1. How to Use This Document

This document is the single reference for the PathFinder Integration Phase. It is designed for three distinct audiences and should be used as follows:

Audience	How to Use
Development Team	Follow sections in order. Each section ends with a clear deliverable. Use data contracts as the interface between components. Stick to the implementation order in Section 9.
AI Collaborators	Read Section 5 (Component Specs) and Section 6 (Data Contracts) first. Each component defines strict input/output boundaries. Do not cross component responsibilities. Technology choices are left to implementation.
Supervisor	Section 2 covers the phase goal and MVP. Section 3 shows the architecture diagram. Section 8 covers the testing strategy. Section 10 covers deployment.

This document does not specify LLM models, embedding models, vector stores, or any specific library versions. Those decisions are made during implementation and recorded separately.

2. Phase Goal & Scope

2.1 Goal Statement

Build the interaction and orchestration backbone of PathFinder — a partial but fully functional end-to-end system that integrates the existing KG engine and RAG engine into a unified advising experience.

The system must:

- Accept natural-language queries from a student user
- Classify query intent and extract relevant entities
- Route to the correct engine(s): KG, RAG, or both
- Incorporate a single curated student profile for student-aware queries
- Maintain session state across multi-turn conversations
- Aggregate results from multiple engines into one coherent answer

2.2 MVP Definition

A successful phase outcome = at least one working, verified example from each of the six user workflows defined in Section 4, with correct routing, student context handling where applicable, and a coherent final response.

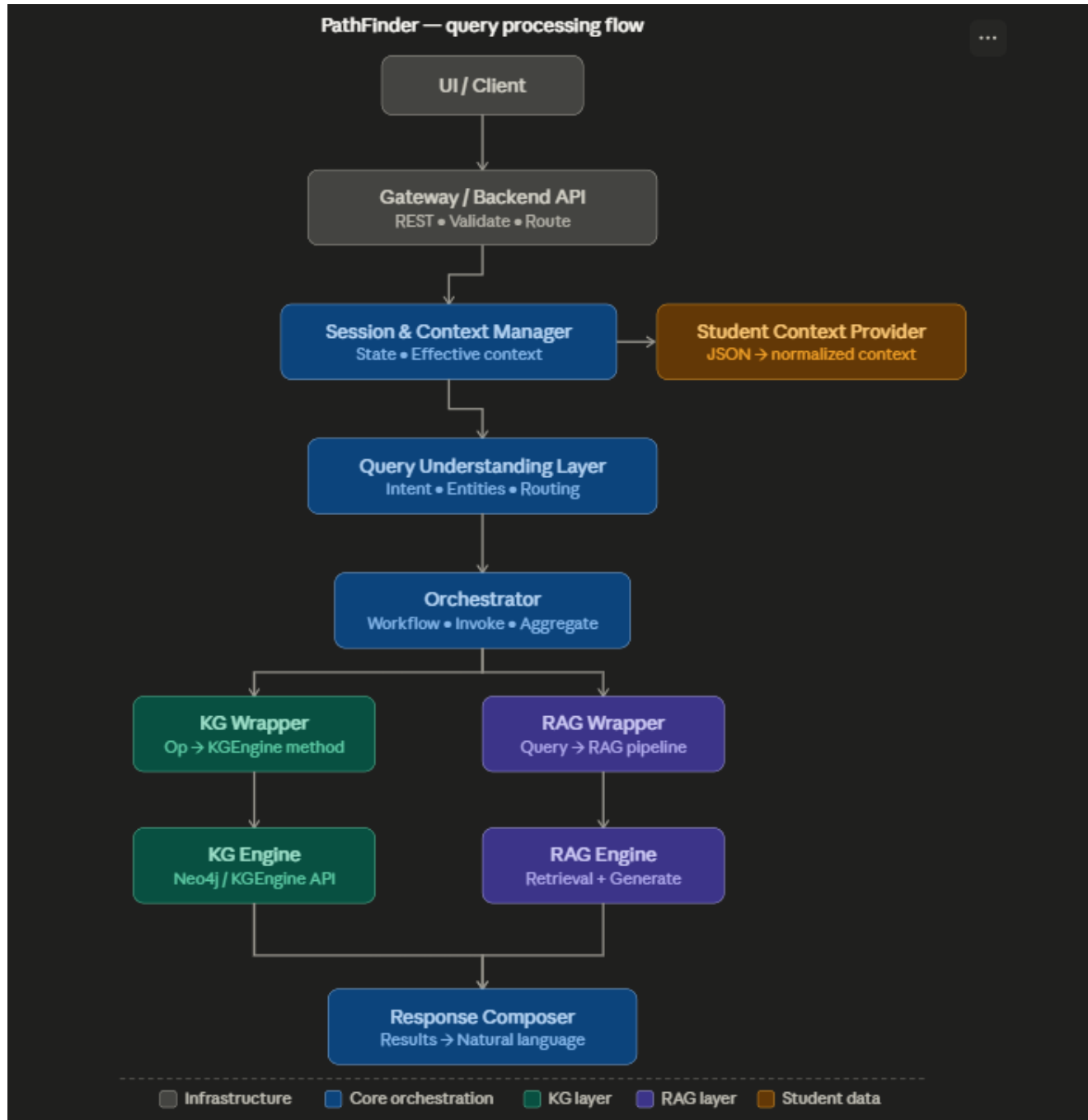
2.3 Scope

☑ In Scope	✗ Out of Scope
• Orchestration between KG and RAG engines • Query understanding and routing logic • Entity extraction and resolution • Session and context management • Curated single-student testing setup • Response aggregation and composition • Minimal working UI (chat interface) • Containerized local deployment	• Academic Logic Engine • Student Analytics Engine • Full student database integration • Predictive modeling • Production-grade deployment • Full dashboard or visualization system • Multi-student scalability • Authentication and access control

3. System Architecture

3.1 High-Level Architecture Diagram

The diagram below shows the complete end-to-end flow of a user request through all system components. Arrows represent the direction of data flow during query processing. The response travels back through the same path in reverse.



3.2 Component Summary

Component	Status	Responsibility
UI / Client	Build Now	Chat interface — sends queries, displays answers
Gateway / Backend API	Build Now	Single entry point, validates requests, passes session ID
Session & Context Manager	Build Now	Loads/saves session state, builds effective student context
Student Context Provider	Build Now	Loads student JSON record, normalizes it for downstream use
Query Understanding Layer	Build Now	Intent classification, entity extraction, engine routing decision
Orchestrator	Build Now	Workflow selection, engine invocation, result aggregation
KG Wrapper	Build Now	Translates structured ops to KGE engine Python API calls
RAG Wrapper	Build Now	Passes sub-query to RAG pipeline, returns answer + citations
KG Engine	Exists	Neo4j-backed deterministic query engine (KGE engine class)
RAG Engine	Exists	Handbook retrieval-augmented generation pipeline
Response Composer	Build Now	Converts aggregated structured results to natural language

4. Supported User Workflows

Every query in this phase maps to one of six defined workflows. The orchestrator selects and executes the appropriate workflow based on structured output from the Query Understanding Layer. All six must have a passing test case for the phase to be complete.

ID	Workflow	Engine(s)	Example Query	Key Requirement
W1	KG Only	KG	"What are the prerequisites of CS-301?"	Entity resolution, KGE engine call, structured result
W2	RAG Only	RAG	"What is the GPA policy for academic warning?"	Retrieval over handbook, cited answer
W3	Mixed (KG + RAG)	KG + RAG	"What courses prepare me for AI and the rules?"	Parallel engine calls, merged response
W4	Student-Aware	KG / RAG / both	"What skills am I missing for Data Scientist?"	Student context injection, gap computation
W5	Multi-turn	Depends	"And what about its prerequisites?" (follow-up)	Session continuity, entity carry-over
W6	Ambiguous / Clarify	None / one	"Tell me about AI"	Ambiguity detection, clarification request

5. Component Specifications

Each component is defined by its role, strict input/output contract, and explicit boundaries. Implementations must not cross these boundaries.

5.1 UI / Client

Attribute	Detail
Role	Thin interface between the student and the backend. Collects user input, displays answers and citations, maintains visual chat thread.
Must Do	Send query + session_id + student_id to gateway • Render natural-language response • Display clickable citations • Show session continuity across turns
Must NOT Do	Call KG or RAG directly • Perform routing or classification • Store student records • Apply business logic
Input	User typed text, active student ID
Output	HTTP POST to /query endpoint of Gateway API
Design Note	Keep it minimal this phase. A simple chat layout is enough. No dashboard, no visualization, no authentication.

5.2 Gateway / Backend API

Attribute	Detail
Role	Single HTTP entry point. Validates incoming request shape, assigns or reads session_id, delegates to the internal pipeline, returns final response.
Must Do	Validate request body • Read or generate session_id • Forward to Session & Context Manager • Return final response body to UI
Must NOT Do	Perform query interpretation • Access KG or RAG directly • Compute routing • Modify session state
Endpoint	One endpoint is sufficient for this phase: POST /query. A single endpoint handles all six workflow types internally — routing logic lives inside the backend, not in the URL. Add more endpoints only if explicitly needed later.
Design Note	Error responses must be structured JSON, not plain text. Keep this component thin — it is a boundary, not logic.

5.3 Session & Context Manager

Attribute	Detail
-----------	--------

Role	Maintains runtime conversational context. Combines the base student record with temporary session-level overrides to produce the effective context used downstream.
Must Do	Load session by session_id • Apply temporary overrides after Query Understanding detects them • Track last referenced entities (course, role, workflow) • Save updated session state after each turn • Expose effective_context to the Query Understanding Layer and Orchestrator
Must NOT Do	Interpret or parse user queries • Detect session overrides in text (this belongs to Query Understanding Layer) • Call engines • Permanently modify the student record • Perform domain reasoning
Core Formula	$\text{effective_context} = \text{base_student_record} + \text{session_overrides}$
Override Flow — Critical	Override detection is performed by the Query Understanding Layer, which includes detected overrides in the session_overrides field of the Structured Query. The Session Manager's job is only to receive and apply those overrides — not to detect them. Sequence: QU detects → Session Manager applies → Orchestrator receives updated effective_context.
Storage (Now)	In-memory dictionary keyed by session_id. Simple and sufficient for a single-instance demo.
Storage (Future)	Redis or persistent key-value store. Swap without changing the interface. Environment variable SESSION_STORE controls which backend is active.

5.4 Student Context Provider

Attribute	Detail
Role	Retrieves the student record and normalizes it into a clean, consistent JSON object for system use. Hides the underlying storage format from all other components.
Must Do	Load student record by student_id • Normalize structure into the canonical Student Context schema (Section 6.3) • Return only the fields the system needs • Handle record not found gracefully
Must NOT Do	Store session data • Permanently modify the student record • Perform reasoning • Call engines
Storage (Now)	Read from a JSON file (one curated student). The JSON file mirrors the canonical schema exactly so no transformation is needed in this phase.
Storage (Future)	Query a database — this component normalizes the DB result to the same JSON schema. Only this component changes.
Key Principle	Decouples storage format from system logic. When switching to a database, only this component changes. All other components always receive the same clean JSON.

5.5 Query Understanding Layer

Attribute	Detail
Role	Transforms raw user text into a structured query representation. This is the only component that interprets natural language for routing purposes. It also detects session overrides embedded in user text.
Must Do	Classify query intent • Determine engine pattern (KG / RAG / Mixed) • Extract named entities (course code, role name, track, skill) • Detect student-awareness flag • Detect ambiguity and return clarification flag when needed • Detect and output session overrides in the session_overrides field of Structured Query (e.g. hypothetical course additions, target role changes)
Must NOT Do	Call KG or RAG engines • Aggregate results • Generate final user response • Apply overrides to session state (this belongs to Session Manager)
Override Detection	When a user says things like “assume I’ve completed CS-401” or “hypothetically, if I add AI-311”, the QU Layer extracts this as a structured override and places it in session_overrides in the output. The Session Manager then applies it. QU detects — Session Manager applies. Never the other way around.
Approach	Hybrid: deterministic rule layer (first pass) + LLM-based fallback (for ambiguous or complex queries). LLM must return structured JSON, not free text.
Critical	Entity IDs must be resolved to exact KG schema IDs (e.g., course_code, role_id). Resolution is the responsibility of this layer.

5.6 Orchestrator (Workflow Coordinator)

Attribute	Detail
Role	Controls the execution of every request. Selects the workflow, invokes the correct engine(s), and aggregates their structured outputs.
Must Do	Map intent to predefined workflow • Invoke KG wrapper, RAG wrapper, or both • Pass clean structured inputs to wrappers • Aggregate results from multiple engines • Handle errors and missing entities gracefully • Forward structured result package to Response Composer
Must NOT Do	Interpret raw user text • Generate final response • Implement KG or RAG logic internally • Access raw databases directly • Use an LLM to decide routing
Design	Rule-governed. Workflows are predefined code paths, not dynamic LLM decisions. One workflow selection function maps engine_pattern → execution path.
Error Cases	Missing entity → return clarification request • Engine error → return structured error payload • Invalid workflow → return structured explanation

5.7 Engine Wrappers (KG & RAG)

Attribute	KG Wrapper	RAG Wrapper
-----------	------------	-------------

Role	Maps structured operation name + parameters → KGEEngine method call	Passes natural-language sub-query → RAG pipeline
Input	operation: string, params: dict, student_context: dict	sub_query: string, student_context: dict (optional)
Output	Structured dict from KGEEngine method	{ answer: string, citations: list }
Must NOT Do	Perform reasoning or routing	Perform reasoning or routing
Note	Calls KGEEngine Python methods directly. See KG component docs for all 15 methods.	Calls the existing RAG pipeline. Treat as a black box; only care about the output contract.

5.8 Response Composer

Attribute	Detail
Role	Converts the aggregated structured result package into a clear, coherent natural-language response. Acts as a presentation layer only.
Must Do	Take original user query + aggregated engine results • Produce fluent, user-facing natural language • Merge KG and RAG outputs when both are present • Include citations from RAG results • Maintain consistent response style
Must NOT Do	Invoke engines • Control workflow • Interpret user intent • Modify or fabricate factual results
Approach	LLM-based composition restricted to formatting and explanation. The LLM receives structured data and is instructed only to present it, not to reason beyond it.
Key Rule	The composer is a presentation layer. It must not alter factual content or add information not present in the structured input.

6. Data Contracts

These are the canonical data structures shared between components. All components must conform to these schemas. They serve as the interface agreement between all implementations.

6.1 API Request & Response

Incoming Request (UI → Gateway)

```
{
  "session_id":      "string | null",    // null = new session
  "user_text":       "string",           // raw natural language query
  "active_student_id": "string"          // e.g. "S_000123"
}
```

Outgoing Response (Gateway → UI)

```
{
  "session_id": "string",                // assigned or existing
  "answer_text": "string",               // final natural language answer
  "citations":  [ { "source": "string", "page": "int | null" } ],
  "status":     "ok | error | clarification_needed"
}
```

6.2 Structured Query (Query Understanding → Orchestrator)

```
{
  "intent":      "string",              // e.g. "get_prerequisites"
  "engine_pattern": "kg | rag | mixed",
  "query_type":  "student_aware | non_student_aware",
  "entities": {
    "course_code": "string | null",
    "role_id":     "string | null",
    "track_id":    "string | null",
    "skill_id":    "string | null"
  },
  "needs_clarification": "boolean",
  "clarification_prompt": "string | null",
  "session_overrides": {
    "added_courses": ["string"], // hypothetical course additions
    "target_role":   "string | null"
  }
}
```

session_overrides in the Structured Query is always populated by the Query Understanding Layer — never by the Session Manager itself. The Session Manager only reads and applies this field. An empty overrides object { } is valid and indicates no override was detected.

6.3 Student Context (Provider → Session Manager → Orchestrator)

The student context schema is derived from the CIS handbook regulations, the course catalogue structure, and the KG engine's function signatures. All fields are either stored directly in the student JSON file or derived from `course_history` at load time.

```
{
  // — Identity
  "student_id":      "string",      // e.g. "S_000123"
  "name":            "string",
  "track_id":        "string",      // e.g. "AI", "CS", "IS"
  "level":           "int",         // academic year: 1-4
  "current_semester": "string",     // "Fall" | "Spring"

  // — Academic standing (handbook-derived)
  "cgpa":            "float",       // 0.0 to 4.0
  "academic_standing": "string",    // "good" | "probation"
  "total_credit_hours_earned": "int", // toward 133 required
  "credit_hours_remaining": "int",   // 133 - earned
  "max_credit_hours_allowed": "int", // 12 | 15 | 18 | 21 (per CGPA)

  // — Course history (source of truth)
  "course_history": [
    {
      "course_code": "string", // exact KG course_code e.g. "C-CS112"
      "course_name": "string",
      "credit_hours": "int",
      "grade": "string | null", // letter grade: A+, A, B+...F, Abs
      "grade_points": "float | null", // 4.0, 3.7, 3.2 ... 0.0
      "semester_taken": "string", // e.g. "Fall 2023"
      "status": "string" // "passed" | "failed" | "in_progress"
    }
  ],

  // — Derived views (computed from course_history at load time)
  "completed_courses": ["string"], // course_codes with status = "passed"
  "failed_courses": ["string"], // course_codes with status = "failed"
  "in_progress_courses": ["string"], // course_codes with status = "in_progress"
  "planned_courses": ["string"] // courses student intends to take next
}
```

`completed_courses`, `failed_courses`, and `in_progress_courses` are derived views. The Student Context Provider computes them from `course_history` at load time. They exist in the context object as pre-computed lists for fast lookup by the KG Engine functions (e.g. `compute_skill_gap`, `find_best_matching_roles`).

6.4 Session State (Stored by Session Manager)

```
{
  "session_id":      "string",
  "active_student_id": "string",
  "last_referenced": {
    "course_code": "string | null",
    "role_id":     "string | null",
    "workflow":    "string | null"
  },
  "overrides": {
    "added_courses": ["string"], // temporary hypothetical additions
  }
}
```

```
    "target_role":      "string | null"
  }
}
```

6.5 Aggregated Result Package (Orchestrator → Response Composer)

```
{
  "original_query": "string",
  "engine_pattern": "kg | rag | mixed",
  "kg_result":      { } | null,          // structured KGEEngine output
  "rag_result": {
    "answer":       "string | null",
    "citations":    [ ]
  } | null,
  "student_context": { } | null,
  "status":         "ok | error | clarification_needed",
  "error_detail":   "string | null"
}
```

7. Curated Test Student Profile

One carefully designed student profile is used for all testing. It covers enough variation to trigger all six workflows and test all student-aware KG operations. Stored as a JSON file and loaded at startup via the Student Context Provider.

7.1 Profile Design Requirements

Requirement	Why It Matters
Has completed a mix of core + elective courses	Enables real skill gap computation from KG engine
CGPA between 2.0 and 3.0 (e.g. 2.85)	Places student in 18 credit-hour limit tier; tests credit-limit RAG queries
Has 1 failed course in history	Tests failed_courses detection; enables retake regulation queries
Has 1-2 courses in progress	Tests in_progress filtering in student-aware workflows
Has a planned course that closes a skill gap	Tests estimate_alignment_improvement KG function
Belongs to a specific track (e.g. AI)	Enables track-specific KG queries
Missing skills for a named career role	Enables skill gap and role recommendation workflows
Total credits around 75-90 (mid-degree)	Tests credit_hours_remaining calculation; not trivially graduating

7.2 Profile JSON Structure

```
// student_profile.json
{
  "student_id": "S_000123",
  "name": "Ahmed Youssef",
  "track_id": "AI",
  "level": 3,
  "current_semester": "Spring",

  "cgpa": 2.85,
  "academic_standing": "good",
  "total_credit_hours_earned": 78,
  "credit_hours_remaining": 55,
  "max_credit_hours_allowed": 18,

  "course_history": [
    {
      "course_code": "C-CS112",
      "course_name": "Introduction to Programming",
      "credit_hours": 3,
      "grade": "A",
      "grade_points": 3.7,
```



```

    "semester_taken": "Fall 2022",
    "status": "passed"
  },
  {
    "course_code": "C-CS201",
    "course_name": "Data Structures",
    "credit_hours": 3,
    "grade": "B+",
    "grade_points": 3.2,
    "semester_taken": "Spring 2023",
    "status": "passed"
  },
  {
    "course_code": "C-AI211",
    "course_name": "Introduction to AI",
    "credit_hours": 3,
    "grade": "C",
    "grade_points": 2.4,
    "semester_taken": "Fall 2023",
    "status": "passed"
  },
  {
    "course_code": "C-CS215",
    "course_name": "Algorithms",
    "credit_hours": 3,
    "grade": "D",
    "grade_points": 1.5,
    "semester_taken": "Spring 2023",
    "status": "failed"
  },
  {
    "course_code": "C-AI311",
    "course_name": "Machine Learning",
    "credit_hours": 3,
    "grade": null,
    "grade_points": null,
    "semester_taken": "Spring 2025",
    "status": "in_progress"
  }
],

"completed_courses": ["C-CS112", "C-CS201", "C-AI211"],
"failed_courses": ["C-CS215"],
"in_progress_courses": ["C-AI311"],
"planned_courses": ["C-AI401", "C-ML321"]
}

```

Replace course codes above with actual codes from the course catalogue CSV. The fields `academic_standing`, `credit_hours_remaining`, and `max_credit_hours_allowed` are pre-computed at load time by the Student Context Provider based on `cgpa` and `total_credit_hours_earned`.

8. Runtime Query Flow

Every user query passes through four sequential phases. This is the canonical execution order — no step may be skipped or reordered.

8.1 Phase Overview

#	Phase	What Happens
1	Preparation	Session Manager loads or creates session state • Student Context Provider loads student record • Session Manager builds effective_context (base + overrides) • Query and effective context pass to Query Understanding Layer
2	Query Understanding	Raw text is classified: intent, engine_pattern, entities, student_aware flag • Any session overrides detected in the text are extracted and placed in session_overrides • Session Manager applies new overrides to update effective_context • Structured query is forwarded to Orchestrator
3	Execution	Orchestrator selects workflow based on engine_pattern • Invokes KG Wrapper, RAG Wrapper, or both • Wrappers call their respective engines • Results return to Orchestrator • Orchestrator aggregates into structured result package
4	Response & Save	Response Composer receives structured result + original query • Generates natural-language response with citations • Session Manager saves updated session state (last_referenced, overrides) • Gateway returns final response body to UI

8.2 Detailed End-to-End Flow Trace

The following traces a complete student-aware query through every component in order. This is the canonical reference for implementation. Example query: “What skills am I missing for Data Scientist?”

```
REQUEST ENTERS
├─ UI sends: { session_id: 'sess_abc', user_text: 'What skills am I missing
               for Data Scientist?', active_student_id: 'S_000123' }
├─ GATEWAY API
│   └─ validates request shape
│   └─ reads session_id ('sess_abc') → passes to Session Manager
├─ SESSION & CONTEXT MANAGER [Phase 1 – Load]
│   └─ loads session by session_id
│       └─ if new: creates { session_id, active_student_id, last_referenced: {},
                             overrides: {} }
│   └─ calls Student Context Provider: get_student('S_000123')
├─ STUDENT CONTEXT PROVIDER
│   └─ reads student_profile.json
│   └─ computes derived views: completed_courses, failed_courses, in_progress
│   └─ returns normalized Student Context object
```

```

→ SESSION & CONTEXT MANAGER [continues]
  └─ applies existing session overrides to base record
  └─ builds effective_context = base_student + session.overrides
      effective_context: { student_id, track_id:'AI', cgpa:2.85,
                          completed_courses:['C-CS112','C-CS201','C-AI211'],
                          ... all student fields ... }
  └─ passes (user_text, effective_context) → Query Understanding Layer

→ QUERY UNDERSTANDING LAYER [Phase 2]
  └─ rule layer: detects 'skills missing' pattern → matches known intent
  └─ extracts entity: 'Data Scientist' → resolves to role_id:'ROLE_DATA_SCIENTIST'
  └─ detects student_aware: TRUE (query says 'I am missing')
  └─ scans for overrides: none found in this query
  └─ returns STRUCTURED QUERY:
      { intent: 'skill_gap_analysis', engine_pattern: 'kg',
        query_type: 'student_aware',
        entities: { role_id: 'ROLE_DATA_SCIENTIST' },
        needs_clarification: false,
        session_overrides: {} // empty - no override detected }

→ SESSION & CONTEXT MANAGER [Phase 2 - Apply Overrides]
  └─ receives Structured Query
  └─ applies session_overrides (empty in this case - no change)
  └─ updates session.last_referenced: { role_id: 'ROLE_DATA_SCIENTIST' }
  └─ passes (structured_query, effective_context) → Orchestrator

→ ORCHESTRATOR [Phase 3]
  └─ reads engine_pattern: 'kg' → selects student_aware_kg_workflow()
  └─ calls KG WRAPPER with:
      { operation: 'compute_skill_gap',
        params: { role_id: 'ROLE_DATA_SCIENTIST',
                  completed_courses: ['C-CS112','C-CS201','C-AI211'] } }

→ KG WRAPPER
  └─ maps operation → KGEEngine.compute_skill_gap(role_id, completed_courses)

→ KG ENGINE (Neo4j / KGEEngine)
  └─ queries Neo4j graph
  └─ returns: { missing_skills: ['Python','SQL','Statistics'],
              covered_skills: ['Data Analysis','ML Basics'],
              gap_percentage: 0.65 }

→ ORCHESTRATOR [aggregates]
  └─ builds result_package:
      { original_query: '...', engine_pattern: 'kg',
        kg_result: { missing_skills: [...], ... },
        rag_result: null, student_context: {...}, status: 'ok' }
  └─ passes result_package → Response Composer

→ RESPONSE COMPOSER [Phase 4]
  └─ receives: original_query + result_package
  └─ builds LLM prompt: structured data + formatting instruction
  └─ calls LLM (external hosted API)
  └─ returns: 'Based on your completed courses, you are missing Python
              proficiency, advanced SQL, and statistical modeling for the Data
              Scientist role. You have covered Data Analysis and ML Basics.'

→ SESSION & CONTEXT MANAGER [Phase 4 - Save]
  └─ updates session.last_referenced: { role_id:'ROLE_DATA_SCIENTIST',
                                         workflow:'student_aware_kg' }
  └─ saves session to in-memory store
  └─ next turn: 'What courses teach those skills?' → Session Manager

```

```
loads sess_abc, sees last_referenced.role_id, resolves context
└─ GATEWAY API
    └─ wraps: { session_id:'sess_abc', answer_text:'...', citations:[], status:'ok' }
    └─ returns HTTP 200 + JSON to UI
└─ UI → displays answer to student
```

Override scenario: if the student had typed 'Assume I also completed C-AI401 — what skills would I still be missing?', the Query Understanding Layer would detect the override and output session_overrides: { added_courses: ['C-AI401'] }. The Session Manager would apply this before the Orchestrator calls compute_skill_gap, so C-AI401 appears in the completed_courses list passed to KGEEngine. The original student record is never modified.

8.3 Error Handling Rules

Failure Case	Expected Behavior
Entity not found in KG	Return structured error: { status: 'error', code: 'ENTITY_NOT_FOUND', message: '...' }. Composer reformats for user.
Ambiguous query	Query Understanding returns needs_clarification: true. Orchestrator skips engine calls. Composer returns clarification question to user.
RAG returns no relevant result	RAG wrapper returns { answer: 'Not found in handbook.', citations: [] }. Composer presents this honestly.
Session not found	Session Manager creates a new session. No error surfaced to user.
Student ID not found	Student Context Provider returns null. Orchestrator downgrades student-aware query to non-student-aware or asks for valid ID.

9. Implementation Order

Build in this exact sequence. Each step is independently testable before the next begins. Do not parallelize steps that share an interface unless both sides are mocked.

Step	Component	What to Build	Done When
1	Test Student JSON	Create student_profile.json with all required fields matching Section 6.3 schema. Verify all 6 workflows can be triggered by this profile.	JSON loads without error; all fields present
2	Student Context Provider	Module that reads JSON by student_id, computes derived views (completed_courses, failed_courses, in_progress_courses), returns normalized Student Context object.	Returns correct schema; handles missing ID
3	Session & Context Manager	In-memory session dict. Functions: get_session, update_session, apply_overrides, build_effective_context. Overrides come IN from Structured Query — not detected here.	Multi-turn session survives 3 consecutive turns
4	Gateway / Backend API	POST /query endpoint. Validates request, assigns session_id, calls internal pipeline stub, returns response.	curl test returns valid response shape
5	KG Wrapper	Translate operation name + params into KGE engine method calls. Test all 15 operations with sample inputs.	All 15 KGE engine methods callable via wrapper
6	RAG Wrapper	Accept sub-query string, call RAG pipeline, return { answer, citations }. Use existing RAG service.	RAG wrapper returns answer + citations for test query
7	Query Understanding Layer	Build rule layer first (keyword matching for known intents). Add LLM fallback for unrecognized queries. Override detection must be included. Output must match Structured Query schema in 6.2.	6 test queries each return correct engine_pattern + any override correctly detected
8	Orchestrator	Implement workflow selector (map engine_pattern → code path). Wire KG/RAG wrappers. Implement aggregation.	All 6 workflows route and return correct result package
9	Response Composer	Build LLM prompt that takes structured result + query and returns natural language. Include citation formatting.	Final response is readable and factually correct
10	UI / Client	Minimal chat interface. Text input, response display, citation links. Connect to Gateway API.	End-to-end test passes for all 6 workflows

11	Integration & Testing	Run full test suite against curated student profile. Verify all 6 workflow types produce correct results per Section 10.	All 6 workflows pass acceptance criteria
----	-----------------------	--	--

10. Testing Strategy

The system is considered working when all six workflow types produce correct, verifiable results on the curated student profile. Testing is manual in this phase but must follow defined acceptance criteria.

10.1 Component-Level Tests

Component	Test Cases
Student Context Provider	Load valid student_id → correct schema returned • Derived views computed correctly • Load invalid student_id → null returned
Session Manager	Create session → persists • Apply override from Structured Query → reflected in effective context • Multi-turn entity carry-over • Override does NOT modify original student record
Query Understanding	One test query per workflow type (W1–W6) • Hypothetical override query → detected in session_overrides field • Ambiguous query → needs_clarification: true • Entity extracted correctly
KG Wrapper	All 15 KGE engine operations invoked with valid inputs • Invalid entity returns error dict, not exception
RAG Wrapper	Handbook query returns answer + citations • Off-topic query returns fallback message
Orchestrator	KG-only, RAG-only, mixed, student-aware: all route correctly • Missing entity returns clarification request
Response Composer	KG-only result → readable sentence • Mixed result → merged paragraph with citations

10.2 End-to-End Acceptance Tests

Test	Query	Expected Engine Route	Accept Criteria
T1	"What are the prerequisites of [course]?"	KG only	Correct prerequisite list returned
T2	"What is the rule for retaking a failed course?"	RAG only	Cited handbook answer returned
T3	"What courses teach Python and what are the rules?"	KG + RAG	Both results merged in one answer
T4	"What skills am I missing for Data Scientist?"	KG + student context	Correct gap list for test student
T5	"What about its prerequisites?" (after T1)	KG only	Resolves course from session; correct prereqs
T6	"Tell me about AI"	None	Clarification question returned to user

11. Deployment Strategy

Scope: This deployment strategy is for LOCAL DEVELOPMENT only. No cloud infrastructure, no production environment, no public access. The goal is for both teammates to run the entire system with a single command on any machine.

All components are containerized using Docker. This ensures consistent behavior across development machines, eliminates dependency conflicts, and simplifies integration between components.

11.1 Guiding Principles

- Simple over clever: each service runs as a single container
- One docker-compose.yml orchestrates all services locally with a single command
- Services communicate over a private Docker network by service name (no IP addresses)
- Environment variables manage all secrets and configuration — no hardcoding
- LLM inference stays EXTERNAL: both Query Understanding (LLM fallback) and Response Composer call a hosted LLM API over the internet. Nothing requiring a GPU is containerized.
- Existing engines (KG, RAG) are containerized as-is and exposed on known internal ports

11.2 What Is and Is Not Containerized

Item	Containerized?	Reason
Gateway (FastAPI backend)	Yes — local container	Core orchestration logic; teammates need identical env
UI / Client	Yes — local container	Clean separation from backend; avoids port conflicts
KG Engine (KGEEngine + Neo4j)	Yes — local container	Neo4j requires its own data volume; isolated cleanly
RAG Engine (retrieval pipeline)	Yes — local container	Has its own model/index dependencies; isolate them
Session store (in-memory)	No — inside gateway	In-memory dict needs no separate service in this phase
LLM for Query Understanding	No — external API	Hosted API; no GPU needed; no Docker overhead
LLM for Response Composer	No — external API	Same reason. API key passed via environment variable
Student profile JSON	No — mounted file	Simple file mount into gateway container via volume

11.3 Service Map

Service	Internal Port	Host Port	Notes
ui	3000	3000	Frontend chat interface
gateway	8000	8000	FastAPI backend entry point
kg-engine	8001	—	KGEngine HTTP wrapper; internal only
rag-engine	8002	—	RAG pipeline; internal only
neo4j	7474 / 7687	7474 / 7687	Neo4j DB. Uses official Neo4j Docker image.

11.4 Docker Compose Structure

```
# docker-compose.yml (structure — not final values)
version: '3.9'

services:
  neo4j:
    image: neo4j:5
    environment:
      - NEO4J_AUTH=neo4j/${NEO4J_PASSWORD}
    volumes:
      - neo4j_data:/data
      - ./data:/import
    ports: ['7474:7474','7687:7687']

  kg-engine:
    build: ./kg_engine
    environment:
      - NEO4J_URI=bolt://neo4j:7687
      - NEO4J_USER=neo4j
      - NEO4J_PASSWORD=${NEO4J_PASSWORD}
    depends_on: [neo4j]
    expose: ['8001']

  rag-engine:
    build: ./rag_engine
    expose: ['8002']

  gateway:
    build: ./gateway
    environment:
      - KG_ENGINE_URL=http://kg-engine:8001
      - RAG_ENGINE_URL=http://rag-engine:8002
      - LLM_API_KEY=${LLM_API_KEY}      # external LLM API
      - SESSION_STORE=memory          # swap to 'redis' when needed
    volumes:
      - ./gateway/data/student_profile.json:/app/data/student_profile.json:ro
    ports: ['8000:8000']
    depends_on: [kg-engine, rag-engine]

  ui:
    build: ./ui
    environment:
      - GATEWAY_URL=http://gateway:8000
    ports: ['3000:3000']
```

```

    depends_on: [gateway]

volumes:
  neo4j_data:

```

11.5 Environment Variables

Never commit .env files to version control. Commit only .env.example with all required keys and empty values. Each teammate creates their own .env locally.

```

# .env.example
NEO4J_PASSWORD=
LLM_API_KEY=          # key for the hosted LLM API (QU fallback + Composer)
SESSION_STORE=memory  # 'memory' | 'redis'

```

11.6 Startup & Teardown

```

# Start all services (builds images if needed)
docker compose up --build

# Stop without removing data
docker compose down

# Full reset including Neo4j data
docker compose down -v

# Rebuild a single service only
docker compose up --build gateway

```

11.7 Recommended Project Structure

```

pathfinder/
├── docker-compose.yml
├── .env.example
├── .gitignore          # must include .env
├── gateway/            # FastAPI backend + all orchestration
│   ├── Dockerfile
│   ├── main.py         # FastAPI app + /query endpoint
│   ├── session_manager.py
│   ├── query_understanding.py
│   ├── orchestrator.py
│   ├── response_composer.py
│   ├── wrappers/
│   │   ├── kg_wrapper.py
│   │   └── rag_wrapper.py
│   └── data/
│       └── student_profile.json
├── kg_engine/          # existing KG codebase
│   └── Dockerfile
├── rag_engine/         # existing RAG codebase
│   └── Dockerfile
└── ui/                 # frontend
    └── Dockerfile

```


12. Open Decisions (To Be Made During Implementation)

The following decisions are deliberately left open. They are recorded here to prevent premature technology choices and to ensure each decision is made at the correct implementation step.

Decision	Options	Decide At
UI framework	Streamlit (fast demo) vs React (scalable long-term)	Step 10 — UI build
LLM provider (Query Understanding fallback)	Any hosted API that returns structured JSON output	Step 7 — QU build
LLM provider (Response Composer)	Same or different provider as QU	Step 9 — Composer build
Session storage	In-memory dict (now) vs Redis (when needed). Controlled by SESSION_STORE env var.	After MVP passes; swap if needed
KG engine exposure	Direct Python import vs internal HTTP service (affects whether KG engine needs its own Dockerfile in this phase)	Step 5 — KG wrapper build
RAG engine exposure	Direct Python import vs internal HTTP service	Step 6 — RAG wrapper build
Entity resolution strategy	Exact string match vs fuzzy match vs embedding-based lookup for entity resolution in QU Layer	Step 7 — QU build

PathFinder — Integration Phase Blueprint

This document is implementation-ready. Decisions listed in Section 12 are intentionally deferred.